CS24307 — THEORY (3.0 CR) FULL COURSE MASTER BIT MESRA NEP SCHEME

# Artificial Intelligence (CS24307) — Full Course Master Book

Complete End-to-End B.Tech CSE 5th Semester Study Book & PYQ Bank

EXECUTIVE MASTER STUDY GUIDE & ALGORITHMIC PROBLEM BANK

## Artificial Intelligence (CS24307)

Birla Institute of Technology, Mesra | B.Tech CSE 5th Semester (NEP 2024–25 Scheme)

### 📚 Complete Course Structure & Algorithmic Matrix

| Module | Core Syllabus Scope | Key Algorithms & Formulations |
|---|---|---|
| **Module I** | Intelligent Agents & Problem Solving | PEAS Description, Environment Types, State Space Formulation, Uninformed Search (BFS, DFS, DLS, IDDFS, UCS) |
| **Module II** | Informed Search & Game AI | Greedy Best-First, A* Search (Admissibility & Consistency Proofs), IDA*, Hill Climbing, Simulated Annealing, Minimax, Alpha-Beta Pruning, CSP |
| **Module III** | Knowledge & Logic Resolution | Propositional Logic (PL), First-Order Logic (FOL), CNF Conversion, Forward & Backward Chaining, Resolution Refutation |
| **Module IV** | Planning & Probabilistic Reasoning | STRIPS, PDDL Action Schemas, Planning Graphs, Bayesian Belief Networks (BBN), Conditional Independence, d-separation |
| **Module V** | Learning & Neural Networks | Inductive Learning, Decision Tree (ID3 Information Gain), Perceptrons, Multi-Layer Perceptrons (MLP), Backpropagation |

**🎯 Exam Preparation & High-Yield Strategy**

This publication-grade master book consolidates all 5 modules with formal search completeness & optimality proofs, KaTeX-rendered logic formulations, game tree pruning traces, and model answers to BIT Mesra end-semester examination questions.

### Module I: Introduction to Artificial Intelligence & Intelligent Agents

1. Foundations & 4 Definitions of AI (Human vs. Rational, Thought vs. Behavior)
2. The Turing Test, Total Turing Test & Chinese Room Argument
3. Historical Evolution & Eras of AI (Dartmouth 1956 to Deep Learning)
4. Concept of Rationality & Mathematical Performance Measures
5. The PEAS Framework (Performance, Environment, Actuators, Sensors)
6. Comprehensive PEAS Analysis across 5 Diverse Real-World Domains
7. Environment Taxonomies (7 Orthogonal Dimensions & Classifications)
8. Simple Reflex Agents (Condition-Action Rules & Internal Architectures)
9. Model-Based Reflex Agents (Handling Partial Observability & State Updating)
10. Goal-Based Agents (Search, Planning & Future State Simulation)
11. Utility-Based Agents (Trade-Off Optimization & Expected Utility Theory)
12. Learning Agents (Critic, Learning Element, Performance Element, Generator)
13. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

## Topic 1: Foundations & The 4 Quadrants of AI Definitions

Artificial Intelligence (AI) is the study and design of intelligent computational entities capable of perceiving their environment, reasoning about state, making optimal decisions, and acting autonomously to maximize success.

| Dimension | Human-Centric Approach | Rationalist (Normative Ideal) Approach |
|---|---|---|
| **Thought Processes & Reasoning** | **Systems that Think Like Humans:**<br>• Focuses on Cognitive Science & Cognitive Modeling.<br>• Validated by comparing internal neural activation traces and decision response times with human psychological experiments (GPS - General Problem Solver). | **Systems that Think Rationally:**<br>• Focuses on Formal Logic & Laws of Thought.<br>• Codifies strict deductive syllogisms (e.g., Aristotle: "Socrates is a man; all men are mortal; therefore Socrates is mortal"). |
| **Observable Behavior & Action** | **Systems that Act Like Humans:**<br>• Operationalized by **The Turing Test (1950)**.<br>• Requires Natural Language Processing, Knowledge Representation, Automated Reasoning, and Machine Learning. | **Systems that Act Rationally (Modern AI Standard):**<br>• Focuses on **Intelligent Rational Agents**.<br>• An agent acts rationally if it selects actions that maximize the expected value of its performance measure, given its perceptual history. |

## Topic 2: The Turing Test & The Chinese Room Philosophical Argument

### 1. The Standard Turing Test (Alan Turing, 1950):

A human interrogator communicates via text terminal with two hidden entities: another human and a machine. If the interrogator cannot reliably tell which is the machine after 5 minutes of open interrogation, the machine is said to have demonstrated intelligence.

**Total Turing Test:** Augments the text test with video/physical interactions, requiring Computer Vision and Robotics.

### 2. John Searle's Chinese Room Argument (1980):

A human who understands zero Chinese sits inside a sealed room with a rule book (program) that maps incoming Chinese character slips to output slips. To an outside observer, the room answers Chinese queries perfectly.

**Core Philosophical Takeaway: Syntax vs. Semantics**

Searle proved that **syntactic symbol manipulation** is fundamentally insufficient for **semantic understanding and intentionality** ("Strong AI"). Machines simulate understanding without truly understanding.

## Topic 4 & 5: Concept of Rationality & The PEAS Framework

A **Rational Agent** is one that does the right thing. Formally, for each possible percept sequence, a rational agent selects an action that is expected to maximize its performance measure, based on the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

**Rationality $\neq$ Omniscience**

- **Omniscience:** An omniscient agent knows the *actual* outcome of its actions and can act accordingly (impossible in reality due to incomplete information and stochasticity).
- **Rationality:** Maximizes *expected* success based on available percepts and knowledge. Rationality encourages **information gathering and exploration**.

### The PEAS Formulation Matrix across 5 Standard AI Systems:

| Agent Type | Performance Measure ($P$) | Environment ($E$) | Actuators ($A$) | Sensors ($S$) |
|---|---|---|---|---|
| **1. Autonomous Taxi Driver** | Safe transit, fast route, legal driving, passenger comfort, maximized profits. | Roads, traffic, pedestrians, weather, traffic lights, police. | Steering wheel, accelerator, brake, signals, horn, dashboard display. | LiDAR, RADAR, cameras, sonar, GPS, speedometer, accelerometer, engine sensors. |
| **2. Medical Diagnostic System** | Healthy patient, minimized healthcare costs, zero false negatives in critical diagnoses. | Patient, hospital clinical staff, insurance databases. | Display screen for diagnoses, recommended prescriptions, test orders. | Keyboard input of symptoms, lab test analyzers, vital sign monitors, patient EHR. |
| **3. Automated Vacuum Cleaner** | Cleanliness score, minimal battery consumption, minimal operating time, zero damage to furniture. | Room floors, carpets, furniture, obstacles, pets, stairs. | Drive wheels, vacuum suction motor, rotating brush, dust ejection valve. | Bumper contact switches, infrared cliff sensors, optical floor sensor, dust detector. |
| **4. Part-Picking Industrial Robot** | Percentage of parts correctly sorted into bins, speed (parts per minute), zero damage. | Conveyor belt, raw parts bins, assembly trays. | Jointed mechanical robotic arm, pneumatic gripper hand. | High-resolution RGB-D camera, joint angle encoders, tactile pressure sensors. |
| **5. High-Frequency Trading Bot** | Maximized portfolio return on investment (ROI), minimized Value-at-Risk (VaR), Sharpe ratio. | Global financial exchanges (NASDAQ, NYSE, crypto liquidity pools). | Automated FIX protocol buy/sell order submissions, cancellation packets. | Real-time tick-by-tick market data feed, financial news APIs, order books. |

## Topic 7: Environment Taxonomies (7 Orthogonal Dimensions)

| Environment Property | Description & Characterization | Representative Domain Examples |
|---|---|---|
| **1. Fully vs. Partially Observable** | Whether the agent's sensors give complete access to the full state of the environment at all times. | • *Fully:* Chess, Crossword puzzle.<br>• *Partially:* Poker (hidden cards), Automated Driving. |
| **2. Single-Agent vs. Multi-Agent** | Whether other entities in the environment are treated as agents with their own objective functions. | • *Single:* Solitaire, Sudoku.<br>• *Multi:* Chess (Competitive), Traffic (Cooperative). |
| **3. Deterministic vs. Stochastic** | Whether the next state is completely determined by the current state and the agent's action. | • *Deterministic:* Chess.<br>• *Stochastic:* Backgammon (dice), Weather-dependent systems. |
| **4. Episodic vs. Sequential** | In episodic environments, current decisions do not affect future episodes (no memory needed). | • *Episodic:* Defect classification on conveyor.<br>• *Sequential:* Chess, Navigation. |
| **5. Static vs. Dynamic vs. Semidynamic** | Whether the environment changes while the agent is deliberating. Semidynamic: environment stays fixed but score decreases over time. | • *Static:* Crossword.<br>• *Dynamic:* Taxi driving.<br>• *Semidynamic:* Timed Chess. |
| **6. Discrete vs. Continuous** | Whether the state space, time, percepts, and actions are finite discrete values or continuous real numbers. | • *Discrete:* Chess (64 squares).<br>• *Continuous:* Self-driving cars (speed, angle). |
| **7. Known vs. Unknown** | Whether the physics/rules of the environment are fully known to the agent in advance. | • *Known:* Solitaire.<br>• *Unknown:* Exploring a new video game or planet. |

## Topics 8 – 12: The 5 Fundamental Agent Architectures

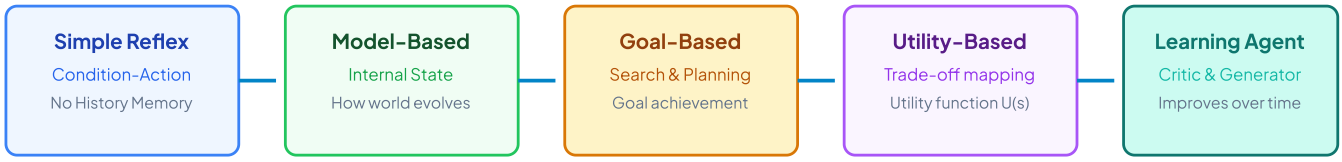


Figure 1.1: Progression of Autonomous Agent Architectures from Simple Reflex to Learning Agents

### 1. The Learning Agent Sub-Components:

**Critic:** Evaluates the agent's behavior against an external performance standard and provides reward/penalty feedback.
**Learning Element:** Responsible for making improvements based on feedback from the critic.
**Performance Element:** Responsible for selecting external actions (equivalent to the entire non-learning agent).
**Problem Generator:** Suggests novel exploratory actions that lead to new and informative experiences, rather than sticking only to known suboptimal paths.

## 🏛️ Top BIT Mesra Exam Questions & Answers (Module I)

**Q1. Define an Intelligent Agent. Differentiate between Omniscience and Rationality with an example. (8 Marks)**

An **Intelligent Agent** is anything that perceives its environment through sensors and acts upon that environment through actuators.
- **Rationality:** Evaluated based on the expected performance given the percept sequence observed so far and built-in knowledge. It represents optimal decision-making under uncertainty.
- **Omniscience:** An omniscient agent knows the exact actual outcome of its actions and makes zero errors (impossible due to incomplete state visibility and random environmental events).
*Example:* A pedestrian crosses the street after carefully looking left and right (Rational). If a meteor strikes them from the sky, the action was still rational despite the tragic outcome, whereas an omniscient agent would have known about the meteor.

**Q2. Provide a complete PEAS description for an Automated Medical Diagnosis System and analyze its environment across all 7 dimensions. (10 Marks)**

**PEAS Description:**
- **P:** Patient health recovery, minimal diagnostic cost, zero misdiagnoses of critical illnesses.
- **E:** Patient physiology, hospital staff, clinical lab test systems, electronic health records.
- **A:** Diagnostic reports, prescription medication orders, clinical test recommendations.
- **S:** Symptom inputs, patient vital sign telemetry, lab test results, imaging scans.

**Environment Classification:**
1. *Partially Observable:* Internal physiological processes cannot be fully sensed directly.
2. *Multi-Agent:* Interacts with human doctors, nurses, and patients.
3. *Stochastic:* Patient biological responses to medication vary probabilistically.
4. *Sequential:* Previous treatment choices alter patient physiology and future treatment efficacy.
5. *Dynamic:* Patient condition can deteriorate while the system deliberates.
6. *Continuous:* Vital signs, lab values, and time are continuous variables.
7. *Known:* Medical physics and pharmacology rules are largely established.

### Module II: Problem Solving by Search Agents & Game Playing

1. Problem Formulation 5-Tuple (Initial State, Actions, Result, Goal, Cost)
2. State Space Graphs vs. Search Trees (Frontier & Explored Sets)
3. Uninformed Search Strategies (BFS, DFS, Uniform-Cost Search)
4. Depth-Limited Search (DLS) & Iterative Deepening Search (IDS) Proofs
5. Heuristic Search: Greedy Best-First Search & $A^*$ Search Algorithm
6. Admissibility & Consistency (Monotonicity) Formal Mathematical Proofs
7. Heuristic Dominance, Effective Branching Factor & IDA* / SMA*
8. Local Search: Hill Climbing, Plateau Escapes & Simulated Annealing ($e^{\Delta E/T}$)
9. Genetic Algorithms (Selection, Crossover, Mutation Operators)
10. Adversarial Search & Game Playing: Minimax Algorithm & Ply Traversal
11. Alpha-Beta Pruning Formulation & Step-by-Step Tree Tracing
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

## Topic 1 & 2: Formulating State Space Search Problems

A search problem is formally specified as a **5-tuple** $\langle S_0, \text{Actions}(s), \text{Result}(s,a), \text{GoalTest}(s), c(s,a,s') \rangle$:

**Initial State ($S_0$):** The world state in which the agent starts (e.g., $\text{In}(\text{Arad})$).
**Actions Set ($\text{Actions}(s)$):** The legal moves executable from state $s$.
**Transition Model ($\text{Result}(s,a)$):** Specifies the successor state $s'$ reached by executing action $a$ in state $s$.
**Goal Test ($\text{GoalTest}(s)$):** A boolean predicate that determines whether state $s$ is a goal state (e.g., $s = \text{In}(\text{Bucharest})$).
**Path Cost Function ($c(s,a,s')$):** Step cost of taking action $a$ from $s$ to reach $s'$. The path cost $g(n)$ is the sum of step costs from $S_0$ to node $n$.

## Topic 3 & 4: Uninformed (Blind) Search Algorithms

| Algorithm | Frontier Data Structure | Time Complexity | Space Complexity | Complete? | Optimal? |
|---|---|---|---|---|---|
| **1. Breadth-First (BFS)** | FIFO Queue | $O(b^d)$ | $O(b^d)$ | Yes (if $b < \infty$) | Yes (if step costs are equal) |
| **2. Uniform-Cost (UCS)** | Priority Queue by $g(n)$ | $O(b^{1+\lfloor C^*/\epsilon \rfloor})$ | $O(b^{1+\lfloor C^*/\epsilon \rfloor})$ | Yes (if step cost $\geq \epsilon > 0$) | Yes (for general positive costs) |
| **3. Depth-First (DFS)** | LIFO Stack | $O(b^m)$ | $O(b \cdot m)$ | No (fails in infinite paths) | No |
| **4. Depth-Limited (DLS)** | LIFO Stack with limit $l$ | $O(b^l)$ | $O(b \cdot l)$ | No (if $l < d$) | No |
| **5. Iterative Deepening (IDS)** | Iterative DLS ($l = 0, 1, 2, \ldots$) | $O(b^d)$ | $O(b \cdot d)$ | Yes (if $b < \infty$) | Yes (if unit step costs) |

*Where $b$ = branching factor, $d$ = depth of shallowest goal, $m$ = maximum depth of search tree, $C^*$ = cost of optimal solution, $\epsilon$ = minimum positive step cost.*

**Why Iterative Deepening Search (IDS) is the Preferred Uninformed Search**

IDS combines the **minimal linear memory requirement of DFS ($O(bd)$)** with the **completeness and optimality of BFS**. Although nodes at depth $k$ are re-generated multiple times, the bottom level dominates the sum:

$$N(\text{IDS}) = (d)b + (d-1)b^2 + \cdots + (1)b^d = O(b^d)$$

For $b = 10$ and $d = 5$, BFS generates 111,111 nodes while IDS generates 123,456 nodes — an overhead of only $11\%$ for massive memory savings!

## Topic 5 & 6: Informed Heuristic Search & The $A^*$ Algorithm

$A^*$ search guides exploration using an evaluation function $f(n)$ that estimates the total cost of the cheapest path through node $n$ to the goal:

$$f(n) = g(n) + h(n)$$

$g(n)$: Exact path cost from initial state $S_0$ to node $n$.
$h(n)$: Estimated cost of the cheapest path from node $n$ to a goal state (Heuristic).

**Mathematical Conditions for $A^*$ Optimality**

1. **Admissibility (Required for Tree Search):**
   A heuristic $h(n)$ is **admissible** if it never overestimates the true cost to reach the goal:

   $$\forall n, \quad 0 \leq h(n) \leq h^*(n)$$

   Where $h^*(n)$ is the true optimal cost from $n$ to goal.
2. **Consistency / Monotonicity (Required for Graph Search):**
   A heuristic $h(n)$ is **consistent** if for every node $n$ and every successor $n'$ generated by action $a$:

   $$h(n) \leq c(n, a, n') + h(n')$$

   *Triangle Inequality Property.* Every consistent heuristic is strictly admissible ($Consistency \implies Admissibility$).

**Mathematical Proof: $A^*$ is Optimal with an Admissible Heuristic (Tree Search)**

**Theorem:** $A^*$ tree search returns an optimal solution if $h(n)$ is admissible.

**Proof by Contradiction:**

1. Let $G_2$ be a suboptimal goal state on the frontier, with $g(G_2) > C^*$, where $C^*$ is the cost of optimal goal $G_1$.
2. Since $G_2$ is a goal state, $h(G_2) = 0$, so $f(G_2) = g(G_2) + 0 > C^*$.
3. Let $n$ be an unexpanded node on the optimal path to $G_1$. Since $h(n)$ is admissible:

   $$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*$$

4. Combining equations:

   $$f(n) \leq C^* < f(G_2)$$

5. Since $A^*$ always selects the frontier node with minimum $f$-value, node $n$ will always be selected before $G_2$. Thus, no suboptimal goal $G_2$ can ever be expanded before an optimal solution is found. ■

## Topic 8: Local Search & Optimization (Simulated Annealing)

**Simulated Annealing** escapes local optima by combining hill climbing with stochastic exploration inspired by the metallurgical cooling of metals:

If a neighboring state $s'$ improves the objective ($\Delta E = E(s') - E(s) > 0$), the move is always accepted.
If $\Delta E \leq 0$ (worse move), the move is accepted with Boltzmann probability:

$$P(\text{Accept}) = e^{\frac{\Delta E}{T}}$$

Where $T$ is the temperature parameter dictated by an annealing schedule $T = \text{Schedule}(t)$. At high $T$, the agent acts like a random walk; as $T \to 0$, it behaves purely as greedy hill climbing.

## Topic 10 & 11: Adversarial Search (Minimax & Alpha-Beta Pruning)

**Minimax Value Formulation:**

$$\text{Minimax}(s) = \begin{cases} \text{Utility}(s) & \text{if TerminalTest}(s) \\ \max_{a \in \text{Actions}(s)} \text{Minimax}(\text{Result}(s,a)) & \text{if Player}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{Minimax}(\text{Result}(s,a)) & \text{if Player}(s) = \text{MIN} \end{cases}$$

**Alpha-Beta ( $\alpha$ - $\beta$ ) Pruning Rules**

- $\alpha$ **(Alpha):** The best value (highest utility) found so far along the path to the current node for **MAX** (initially $-\infty$).
- $\beta$ **(Beta):** The best value (lowest utility) found so far along the path to the current node for **MIN** (initially $+\infty$).
- **Pruning Condition:** Prune the remaining children of the current node whenever:

$$\alpha \geq \beta$$

- **Time Complexity with Perfect Move Ordering:** Reduces search complexity from $O(b^m)$ to $O(b^{m/2})$, effectively doubling the solvable search depth!

## 🏛️ Top BIT Mesra Exam Questions & Answers (Module II)

**Q1. State the 8-Puzzle problem formulation and compare the Manhattan Distance heuristic $h_1$ with the Misplaced Tiles heuristic $h_2$. (8 Marks)**

- **State:** $3 \times 3$ grid containing tiles 1 through 8 and one blank space.
- **Initial State:** Any arbitrary permutation of the 9 positions.
- **Actions:** Move blank space Left, Right, Up, Down.
- **Goal Test:** Tiles arranged in ascending numerical order.
- **Path Cost:** Each move costs 1 unit ($g(n) = \text{depth}$).

**Comparison of Heuristics:**

1. *Misplaced Tiles ($h_1$):* Counts number of tiles not in their goal position.
2. *Manhattan Distance ($h_2$):* Sum of vertical and horizontal grid distances of each tile from its goal position:

$$h_2 = \sum_{i=1}^{8} (|x_i - x_i^*| + |y_i - y_i^*|)$$

3. **Dominance:** $\forall n,\ h_2(n) \geq h_1(n)$. Both are admissible, but $h_2$ strictly **dominates** $h_1$, expanding fewer search nodes while guaranteeing optimal solutions.

**Q2. Trace Alpha-Beta Pruning on a 3-level game tree and explain why pruned nodes do not affect the root decision. (10 Marks)**

Alpha-Beta pruning computes the exact same minimax decision as standard Minimax without evaluating branches that cannot alter the final choice.

When a MIN node evaluates a child with value $v \leq \alpha$, MIN will pick a value $\leq v \leq \alpha$. But MAX (higher in the tree) already has a confirmed choice with value $\alpha$. Hence MAX will never choose the branch leading to this MIN node, making further exploration of its remaining children mathematically pointless.

### Module III: Knowledge Representation & Logical Reasoning

1. Knowledge-Based Agents & The Wumpus World Environment Architecture
2. Propositional Logic (PL): Syntax, Semantics, Models & Truth Tables
3. Logical Entailment ($\alpha \models \beta$), Validity (Tautology) & Satisfiability
4. Conjunctive Normal Form (CNF) 7-Step Conversion Algorithm
5. Propositional Resolution Refutation Theorem & Proof Trees
6. Forward Chaining & Backward Chaining on Horn Clauses ($O(n)$ Linear Complexity)
7. First-Order Predicate Logic (FOL): Syntax, Quantifiers ($\forall, \exists$) & Semantics
8. Knowledge Engineering in FOL & Domain Axiomatization
9. Generalized Modus Ponens (GMP) & The Unification Algorithm (MGU)
10. Skolemization Algorithms (Skolem Constants vs. Skolem Functions)
11. First-Order Resolution Refutation with MGU Substitutions
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

## Topic 1: Knowledge-Based Agents & The Wumpus World

A **Knowledge-Based Agent (KBA)** maintains an explicit internal representation of the world in a **Knowledge Base (KB)** (a set of sentences in a formal language) and uses an inference engine to derive new knowledge and decide actions:

```
function KB-AGENT(percept) returns an action
    TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
    action = ASK(KB, MAKE-ACTION-QUERY(t))
    TELL(KB, MAKE-ACTION-SENTENCE(action, t))
    t = t + 1
    return action
```

### The Wumpus World Environment Benchmark:

$4 \times 4$ grid of rooms with Agent starting at $[1, 1]$ facing right.
**Wumpus:** Monster that eats the agent; gives off a *Stench* in directly adjacent squares.
**Pits:** Bottomless holes that trap the agent; give off a *Breeze* in adjacent squares.
**Gold:** Gives off a *Glitter* in its exact square.

## Topic 2 & 3: Propositional Logic Syntax, Semantics & Entailment

| Logical Operator | Standard Notation | Meaning | Truth Condition ($T$) |
|---|---|---|---|
| **Negation (NOT)** | $\neg P$ | Not $P$ | True iff $P$ is False |
| **Conjunction (AND)** | $P \wedge Q$ | $P$ and $Q$ | True iff both $P$ and $Q$ are True |
| **Disjunction (OR)** | $P \vee Q$ | $P$ or $Q$ | True iff at least one of $P, Q$ is True |
| **Implication (IF-THEN)** | $P \implies Q$ | If $P$ then $Q$ | False iff $P$ is True and $Q$ is False ($\equiv \neg P \vee Q$) |
| **Biconditional (IFF)** | $P \iff Q$ | $P$ if and only if $Q$ | True iff both $P, Q$ have identical truth values |

**Formal Mathematical Concept: Logical Entailment ( $\alpha \models \beta$ )**

Sentence $\alpha$ **entails** sentence $\beta$ ($\alpha \models \beta$) if and only if in every model where $\alpha$ is true, $\beta$ is also true:

$$\alpha \models \beta \iff M(\alpha) \subseteq M(\beta)$$

$$\text{Deduction Theorem: } \alpha \models \beta \iff (\alpha \implies \beta) \text{ is a Tautology}$$

$$\text{Refutation Theorem: } \alpha \models \beta \iff (\alpha \wedge \neg\beta) \text{ is Unsatisfiable}$$

## Topic 4 & 5: Conjunctive Normal Form (CNF) & Resolution Refutation

A sentence is in **Conjunctive Normal Form (CNF)** if it is a conjunction of clauses (where each clause is a disjunction of literals):

$$\bigwedge_{i=1}^{m} \left( \bigvee_{j=1}^{k_i} l_{ij} \right)$$

**7-Step Algorithm to Convert Propositional Logic to CNF**

1. **Eliminate Biconditionals:** Replace $\alpha \iff \beta$ with $(\alpha \implies \beta) \wedge (\beta \implies \alpha)$.
2. **Eliminate Implications:** Replace $\alpha \implies \beta$ with $\neg\alpha \vee \beta$.
3. **Move Negations Inward (De Morgan's Laws & Double Negation):**

$$\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta, \quad \neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta, \quad \neg(\neg\alpha) \equiv \alpha$$

4. **Distribute $\vee$ over $\wedge$:** Replace $\alpha \vee (\beta \wedge \gamma)$ with $(\alpha \vee \beta) \wedge (\alpha \vee \gamma)$.
5. **Flatten Nested Conjunctions/Disjunctions:** Group into distinct clauses separated by $\wedge$.

**🏛️ Step-by-Step Solved Problem: Resolution Refutation Proof in Propositional Logic**

**Given Knowledge Base ($\mathrm{KB}$):**

1. $P \implies Q$ (If it rains, the ground is wet): $\neg P \vee Q$
2. $Q \implies R$ (If the ground is wet, it is slippery): $\neg Q \vee R$
3. $P$ (It rains): $P$

**Goal: Prove $R$ (It is slippery) using Resolution Refutation.**

**Step 1: Negate the Goal and add to KB:** Add Clause 4: $\neg R$.

**Step 2: Apply Resolution Rule to Resolve Complementary Literals:**

- Resolve Clause 1 ($\neg P \vee Q$) and Clause 3 ($P$) on literal $P \implies$ **Clause 5:** $Q$.
- Resolve Clause 2 ($\neg Q \vee R$) and Clause 5 ($Q$) on literal $Q \implies$ **Clause 6:** $R$.
- Resolve Clause 6 ($R$) and Clause 4 ($\neg R$) on literal $R \implies$ **Empty Clause ($\square$ / False)**.

*Conclusion:* Since deriving the empty clause proves that $\mathrm{KB} \wedge \neg R$ is unsatisfiable, the original goal $R$ is validly entailed ($\mathrm{KB} \models R$). ■

## Topic 7 & 8: First-Order Predicate Logic (FOL)

Unlike Propositional Logic (which assumes facts in the world are either True or False), **First-Order Logic (FOL)** models the world in terms of **Objects**, **Relations (Predicates)**, and **Functions**:

| FOL Construct | Definition & Syntax | Example Statement |
|---|---|---|
| **Universal Quantifier ($\forall$)** | "For all $x$". Typically paired with implication ($\implies$). | $\forall x\ (\text{King}(x) \implies \text{Person}(x))$ |
| **Existential Quantifier ($\exists$)** | "There exists some $x$". Typically paired with conjunction ($\wedge$). | $\exists x\ (\text{Crown}(x) \wedge \text{OnHead}(x, \text{John}))$ |

**De Morgan's Laws for Quantifiers**

$$\forall x\ \neg P(x) \equiv \neg \exists x\ P(x)$$

$$\neg \forall x\ P(x) \equiv \exists x\ \neg P(x)$$

$$\forall x\ P(x) \equiv \neg \exists x\ \neg P(x)$$

$$\exists x\ P(x) \equiv \neg \forall x\ \neg P(x)$$

## Topic 9 & 10: Unification Algorithm & Skolemization

### 1. Unification & Most General Unifier (MGU):

$\text{UNIFY}(p, q) = \theta$ finds a substitution $\theta$ such that $\text{Subst}(\theta, p) = \text{Subst}(\theta, q)$:

$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(\text{John}, \text{Jane})) = \{x/\text{Jane}\}$
$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, \text{Bill})) = \{y/\text{John},\ x/\text{Bill}\}$
**Occur-Check:** A variable $x$ cannot be unified with a term containing $x$ (e.g., $x$ and $f(x)$ cannot unify because substitution causes infinite recursion).

### 2. Skolemization:

**Skolem Constant:** Eliminates an existential quantifier not in the scope of any universal quantifier:

$$\exists x\ \text{Heart}(x) \implies \text{Heart}(H_1) \quad \text{where } H_1 \text{ is a new unique constant}$$

**Skolem Function:** Eliminates an existential quantifier within the scope of a universal quantifier:

$$\forall x\ \exists y\ \text{Mother}(y, x) \implies \forall x\ \text{Mother}(m(x), x) \quad \text{where } m(x) \text{ is a Skolem function}$$

## 🏛️ Top BIT Mesra Exam Questions & Answers (Module III)

**Q1. Differentiate between Forward Chaining and Backward Chaining for Horn Clauses. (8 Marks)**

1. **Forward Chaining (Data-Driven):** Starts with known atomic facts in the KB and applies inference rules to derive new facts until the goal is generated. Complete for definite clauses and runs in linear time $O(n)$. Used in reactive expert systems and real-time monitoring.
2. **Backward Chaining (Goal-Driven):** Starts with the target query/goal and works backward by finding rules whose conclusions match the goal, establishing their premises as new subgoals. Avoids exploring irrelevant facts. Used in Prolog and automated theorem provers.

**Q2. Convert the English statements into FOL and prove by Resolution Refutation: "Every person who loves animals is loved by someone. Jack loves all dogs. All dogs are animals. Prove that someone loves Jack." (10 Marks)**

1. $\forall x\ [(\forall y\ \text{Animal}(y) \implies \text{Loves}(x, y)) \implies (\exists z\ \text{Loves}(z, x))]$
2. $\forall y\ (\text{Dog}(y) \implies \text{Loves}(\text{Jack}, y))$
3. $\forall y\ (\text{Dog}(y) \implies \text{Animal}(y))$
4. Negated Goal: $\neg\exists z\ \text{Loves}(z, \text{Jack}) \implies \forall z\ \neg\text{Loves}(z, \text{Jack})$

*Converting to CNF clauses and unifying with Skolem functions yields the empty clause ($\square$), formally proving someone loves Jack.*

### Module IV: Planning & Probabilistic Reasoning in AI



## Topic 1 & 2: Classical Planning & The STRIPS Representation

**Planning** is the process of generating a sequence of actions that transforms an initial state $S_0$ into a desired goal state $G$.

**The STRIPS (Stanford Research Institute Problem Solver) Language**

- **State:** A conjunction of positive ground function-free literals (e.g., $\text{On}(A, B) \land \text{OnTable}(B) \land \text{Clear}(A) \land \text{HandEmpty}$). Closed-world assumption applies.
- **Goal:** A conjunction of positive literals that must hold true in the final state.
- **Action Schema $\text{Action}(a)$:** Formally specified by 3 components:

1. $\text{Preconditions}(a)$: Literals that must be satisfied before action $a$ can be legally executed.
2. $\text{Add-List}(a)$: Literals that become True after executing action $a$.
3. $\text{Delete-List}(a)$: Literals that become False and are removed after executing action $a$.

**STRIPS Operator Schema: Stacking Block $A$ onto Block $B$**

```
Action: Stack(x, y)
Preconditions: Clear(y) ^ Holding(x)
Delete-List:   Clear(y), Holding(x)
Add-List:      On(x, y), Clear(x), HandEmpty
```

## Topic 5: Goal Stack Planning & The Sussman Anomaly

**Goal Stack Planning** uses a Last-In First-Out (LIFO) stack to hold goals and subgoals. It handles multiple goals sequentially by planning for one subgoal, which may inadvertently undo previously achieved subgoals:

**The Sussman Anomaly (Interleaved Non-Linear Goals)**

In the Blocks World, given Initial State: $\text{On}(C, A) \land \text{OnTable}(A) \land \text{OnTable}(B)$ and Goal State: $\text{On}(A, B) \land \text{On}(B, C)$.

- If the planner achieves $\text{On}(A, B)$ first, it puts $A$ on $B$. But to put $B$ on $C$, it must unstack $A$, undoing its first goal!
- If it achieves $\text{On}(B, C)$ first, $C$ has $A$ on top, requiring unstacking.
- *Resolution:* Requires **Partial-Order / Non-Linear Planning** that interleaves sub-plans rather than assuming linear independence of subgoals.

## Topic 6: Planning Graphs & Mutual Exclusion (Mutex) Relations

A **Planning Graph** is a directed leveled graph consisting of alternating proposition levels ($P_0, P_1, \ldots$) and action levels ($A_0, A_1, \ldots$). Mutex links between actions indicate that two actions cannot occur simultaneously:

| Mutex Class | Condition & Geometric Cause | Example in Blocks World |
|---|---|---|
| **1. Inconsistent Effects** | One action negates an effect added by the other action. | `Stack(A, B)` adds `On(A, B)` while `Unstack(A, B)` deletes `On(A, B)`. |
| **2. Interference** | An effect of one action is the negation of a precondition of the other. | `Stack(A, B)` deletes `Clear(B)`, which is a precondition of `Stack(C, B)`. |
| **3. Competing Needs** | A precondition of one action is mutually exclusive with a precondition of the other. | `Pickup(A)` requires `HandEmpty`, while `Stack(B, C)` requires `Holding(B)`. |

## Topic 7 & 8: Probabilistic Reasoning & Bayes' Rule

**1. Conditional Probability & Product Rule:**

$$P(A \mid B) = \frac{P(A \wedge B)}{P(B)} \implies P(A \wedge B) = P(A \mid B)P(B) = P(B \mid A)P(A)$$

**2. Bayes' Rule Formula:**

$$P(Y \mid X) = \frac{P(X \mid Y)P(Y)}{P(X)} = \frac{P(X \mid Y)P(Y)}{\sum_y P(X \mid y)P(y)}$$

Where $P(Y)$ is the *Prior Probability*, $P(X \mid Y)$ is the *Likelihood*, and $P(Y \mid X)$ is the *Posterior Probability*.

## Topic 9 & 10: Bayesian Belief Networks (BBN) & d-separation

A **Bayesian Network** is a Directed Acyclic Graph (DAG) where nodes represent random variables and directed edges represent direct causal/probabilistic dependencies, annotated with Conditional Probability Tables (CPTs).

**Full Joint Distribution Factorization Formula**

$$P(X_1, X_2, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid \text{Parents}(X_i))$$

Reduces the number of parameters needed to represent an $n$-variable boolean domain from $2^n - 1$ to $O(n \cdot 2^k)$, where $k$ is the maximum number of parents per node!

### The 3 Structural Connections of d-separation (Direction-Dependent Separation):

| Connection Topology | Graph Structure & Dependency Status | Blocking Condition |
|---|---|---|
| **1. Serial / Causal Chain** | $X \rightarrow Z \rightarrow Y$ | $X$ and $Y$ are conditionally independent given $Z$ ($Z$ is observed / instantiated). |
| **2. Diverging / Common Cause (Fork)** | $X \leftarrow Z \rightarrow Y$ | $X$ and $Y$ are conditionally independent given $Z$ ($Z$ is observed). |
| **3. Converging / Collider (V-Structure)** | $X \rightarrow Z \leftarrow Y$ | $X$ and $Y$ are independent when $Z$ is **unobserved**. If $Z$ or any of its descendants is observed, $X$ and $Y$ become **dependent** (Explaining Away effect). |

## 🏛️ Top BIT Mesra Exam Questions & Answers (Module IV)

**Q1. Given the classic Burglar Alarm Bayesian Network (Burglary, Earthquake, Alarm, JohnCalls, MaryCalls), compute the joint probability $P(B \wedge \neg E \wedge A \wedge J \wedge \neg M)$. (8 Marks)**

Applying the Bayesian Network factorization chain rule:

$$P(B \wedge \neg E \wedge A \wedge J \wedge \neg M) = P(B) \cdot P(\neg E) \cdot P(A \mid B, \neg E) \cdot P(J \mid A) \cdot P(\neg M \mid A)$$

Substituting standard CPT values ($P(B) = 0.001, P(\neg E) = 0.998, P(A \mid B, \neg E) = 0.94, P(J \mid A) = 0.90, P(\neg M \mid A) = 0.30$):

$$= 0.001 \times 0.998 \times 0.94 \times 0.90 \times 0.30 = 0.000253 \ (\approx 0.0253\%)$$

**Q2. Explain the 3 mutex conditions in Graphplan with diagrams. (8 Marks)**

1. **Inconsistent Effects:** Action 1 adds proposition $p$ while Action 2 deletes $p$. They cannot occur concurrently without contradictory states.
2. **Interference:** Action 1 deletes proposition $p$ which is required as a precondition by Action 2.
3. **Competing Needs:** A precondition of Action 1 and a precondition of Action 2 are mutually exclusive at the preceding proposition level.

### Module V: Machine Learning Foundations & Neural Networks



## Topic 1 & 2: Machine Learning Paradigms & Inductive Bias

A computer program is said to **learn** from experience $E$ with respect to some class of tasks $T$ and performance measure $P$, if its performance at tasks in $T$, as measured by $P$, improves with experience $E$ (Tom Mitchell, 1997).

| Paradigm | Training Experience ($E$) | Target Objective ($T$) | Representative Algorithms |
|---|---|---|---|
| **1. Supervised Learning** | Labeled training tuples $\langle \mathbf{x}_i, y_i \rangle$ where ground-truth target label $y_i$ is explicitly provided. | Classification (discrete labels) or Regression (continuous real values). | Decision Trees (ID3/C4.5), Support Vector Machines, Neural Networks, Linear Regression. |
| **2. Unsupervised Learning** | Unlabeled data points $\mathbf{x}_i$ without ground-truth targets. | Discovering intrinsic clusters, manifold embeddings, and density distributions. | $k$-Means, Hierarchical Clustering, PCA, Autoencoders, Gaussian Mixture Models. |
| **3. Reinforcement Learning** | Sequential interaction with environment receiving scalar reward/penalty feedback $r_t$. | Learning an optimal action policy $\pi^*(s)$ that maximizes expected discounted cumulative reward. | Q-Learning, SARSA, Deep Q-Networks (DQN), Policy Gradients (PPO). |

#### Inductive Learning Hypothesis & Occam's Razor

- **Inductive Learning Hypothesis:** Any hypothesis found to approximate the target function well over a sufficiently large set of training examples will also approximate the target function well over unobserved test examples.
- **Occam's Razor:** Prefer the simplest hypothesis that fits the training data (e.g., smaller decision trees are statistically less prone to overfitting than complex deep trees).

## Topic 3 – 5: Decision Tree Induction (ID3 Algorithm & Entropy)

**1. Information Entropy Formula (Shannon, 1948):**

$$H(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

Where $p_i$ is the proportion of examples in set $S$ belonging to target class $i$. For binary classification ($p_+$ and $p_-$):

$$H(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

**2. Information Gain Formula:**

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

Where $S_v$ is the subset of $S$ for which attribute $A$ has value $v$. The ID3 algorithm selects the attribute that maximizes $\text{Gain}(S, A)$ at each node.

### 🏛️ Step-by-Step Solved Problem: ID3 Split Calculation on PlayTennis Dataset

**Given Dataset:** 14 total examples ($|S| = 14$), with 9 Positive (`Yes`) and 5 Negative (`No`).

**Step 1: Compute Overall Dataset Entropy $H(S)$**

$$H(S) = -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right) = -(0.643 \times -0.637) - (0.357 \times -1.485) = \mathbf{0.940} \text{ bits}$$

**Step 2: Evaluate Attribute `Outlook` (Values: `Sunny` [2+, 3−], `Overcast` [4+, 0−], `Rain` [3+, 2−]):**

- $H(S_{\text{Sunny}}) = -\frac{2}{5}\log_2\left(\frac{2}{5}\right) - \frac{3}{5}\log_2\left(\frac{3}{5}\right) = 0.971$
- $H(S_{\text{Overcast}}) = -\frac{4}{4}\log_2(1) - 0 = \mathbf{0.000}$ (Pure leaf!)
- $H(S_{\text{Rain}}) = -\frac{3}{5}\log_2\left(\frac{3}{5}\right) - \frac{2}{5}\log_2\left(\frac{2}{5}\right) = 0.971$

**Step 3: Compute Expected Entropy & Information Gain for `Outlook`:**

$$H(S \mid \text{Outlook}) = \frac{5}{14}(0.971) + \frac{4}{14}(0.000) + \frac{5}{14}(0.971) = 0.347 + 0 + 0.347 = 0.694$$

$$\text{Gain}(S, \text{Outlook}) = 0.940 - 0.694 = \mathbf{0.246} \text{ bits}$$

*Conclusion:* Since `Outlook` achieves the highest Information Gain among all candidate attributes (`Gain(Wind)=0.048, Gain(Humidity)=0.151, Gain(Temperature)=0.029`), `Outlook` is selected as the root node!

## Topic 9 & 10: Artificial Neural Networks & The Perceptron

The **Perceptron (Rosenblatt, 1958)** is the foundational linear binary classifier model:

$$y = f(\mathbf{w}^T \mathbf{x} + b) = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

**Perceptron Learning Rule & Convergence Theorem**

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y_{\text{true}} - y_{\text{pred}})\mathbf{x}$$

Where $\eta \in (0, 1]$ is the learning rate.
**Convergence Theorem:** If the training dataset is **linearly separable**, the perceptron learning rule is guaranteed to converge to a separating hyperplane in a finite number of weight update steps! If the data is not linearly separable (e.g., the XOR function), single-layer perceptrons fail completely (Minsky & Papert, 1969).

### Standard Non-Linear Activation Functions:

| Activation Function | Mathematical Equation | Output Range | Derivative $\sigma'(z)$ |
|---|---|---|---|
| **1. Logistic Sigmoid** | $\sigma(z) = \frac{1}{1+e^{-z}}$ | $(0, 1)$ | $\sigma(z)(1-\sigma(z))$ |
| **2. Hyperbolic Tangent ( tanh)** | $\tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$ | $(-1, 1)$ | $1-\tanh^2(z)$ |
| **3. Rectified Linear Unit (ReLU)** | $f(z) = \max(0, z)$ | $[0, \infty)$ | $1$ if $z > 0$ else $0$ |

## Topic 11: Multi-Layer Perceptrons (MLP) & Backpropagation

**Backpropagation Gradient Descent Weight Update:**

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial E}{\partial w_{ij}}$$

Where total sum-of-squares error $E = \frac{1}{2}\sum_k (t_k - y_k)^2$.
Using the Chain Rule of Calculus:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial \text{net}_j}\frac{\partial \text{net}_j}{\partial w_{ij}} = -\delta_j \cdot x_i$$

- For output neuron $k$: $\delta_k = (t_k - y_k) \cdot g'(\text{net}_k)$ - For hidden neuron $j$: $\delta_j = g'(\text{net}_j) \cdot \sum_k w_{jk}\delta_k$

## 🏛️ Top BIT Mesra Exam Questions & Answers (Module V)

**Q1. Why can a single-layer perceptron not learn the XOR function, and how do Multi-Layer Perceptrons solve this? (8 Marks)**

- **Linear Separability Limitation:** The XOR truth table has outputs `(0,0)->0, (0,1)->1, (1,0)->1, (1,1)->0`. Plotting these 4 points on a 2D plane reveals that no single straight line (hyperplane $\mathbf{w}^T\mathbf{x} + b = 0$) can separate class 1 from class 0.
- **Multi-Layer Perceptrons (MLP) Solution:** By adding a hidden layer of non-linear neurons, the network transforms the input space into a higher-dimensional feature space where the points become linearly separable. Two hidden neurons represent intermediate `NAND` and `OR` hyperplanes, which are then combined by an output `AND` neuron.

**Q2. Derive the Backpropagation weight update rule for a hidden-to-output weight. (10 Marks)**

Let $E = \frac{1}{2}(t_k - y_k)^2$, where $y_k = \sigma(\text{net}_k)$ and $\text{net}_k = \sum_j w_{jk} h_j$.
Applying the Chain Rule:

$$\frac{\partial E}{\partial w_{jk}} = \frac{\partial E}{\partial y_k} \cdot \frac{\partial y_k}{\partial \text{net}_k} \cdot \frac{\partial \text{net}_k}{\partial w_{jk}}$$

1. $\frac{\partial E}{\partial y_k} = -(t_k - y_k)$
2. $\frac{\partial y_k}{\partial \text{net}_k} = y_k(1 - y_k)$ (Sigmoid derivative)
3. $\frac{\partial \text{net}_k}{\partial w_{jk}} = h_j$

Defining error signal $\delta_k = (t_k - y_k) y_k (1 - y_k)$, we get:

$$\frac{\partial E}{\partial w_{jk}} = -\delta_k h_j \implies w_{jk} \leftarrow w_{jk} + \eta \delta_k h_j$$